

UQ-QNN Rerun Experiments

Memristor placement, multi-memristor tests, and backend speedups

Project Presentation

April 8, 2026

Repository Goal

The repository studies uncertainty-aware photonic QNNs built on Clements meshes and memristive photonic circuits.

Current goals:

- train photonic models reproducibly,
- compare standard and memristive circuits,
- quantify uncertainty and calibration,
- make simulator reruns fast enough for architecture studies.

Development Stages

Recent progress happened in four layers:

- 1 **Experiment workflow:** explicit config dictionaries and the Experiment API.
- 2 **Repository cleanup:** PhotonicCircuit + CircuitConfig, public helpers, structured simulation package.
- 3 **Correctness:** fixes for noise handling, memristive UQ, and gradient edge cases.
- 4 **Performance:** fast NumPy memristive backend for discrete and swipe singles runs.

This enabled the rerun experiments shown in the following slides.

Config and readout (current API)

- `input_state`: occupation vector of length `n_modes`; photon number $N = \sum_i \text{input_state}_i$.
- Singles: $N = 1$. Coincidence: $N \geq 2$; channels are N -tuples from `working_detectors` ($C(W, N)$ outcomes, postselected).
- Coincidence regression: `target_mode` is an N -tuple of distinct detector indices. CE: `n_classes` = $C(W, N)$.
- PSR uses $2N$ shift terms per phase (same N). NumPy and Perceval agree on the N -fold layout.

See `CHANGELOG.md` (2026-04-08) and root `README.md` for migration notes.

We focus on four reruns:

- function-vs-memristor comparison across four synthetic targets,
- smooth-step single-memristor placement sweep,
- smooth-step one-vs-two memristor comparison,
- backend benchmark for the new memristive NumPy path.

All results were saved under `reports/` and re-used here.

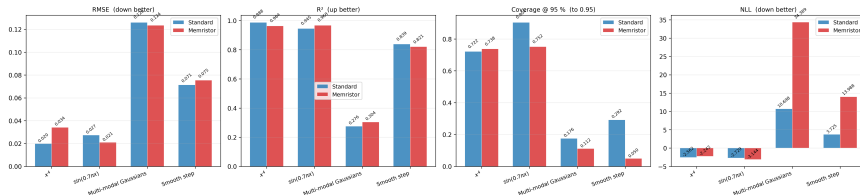
Function Comparison: Main Takeaway

Function	Std RMSE	Mem RMSE	Std R^2	Mem R^2
Quartic	0.0200	0.0342	0.9876	0.9637
Sinusoid	0.0274	0.0210	0.9455	0.9680
Multi-modal	0.1261	0.1236	0.2757	0.3043
Smooth step	0.0714	0.0754	0.8393	0.8209

- Memristors are **not uniformly beneficial**.
- They help on sinusoid and marginally on multi-modal fitting.
- They hurt on quartic and smooth step.

Function Comparison: Visual Summary

Performance Metrics: Standard vs Memristor



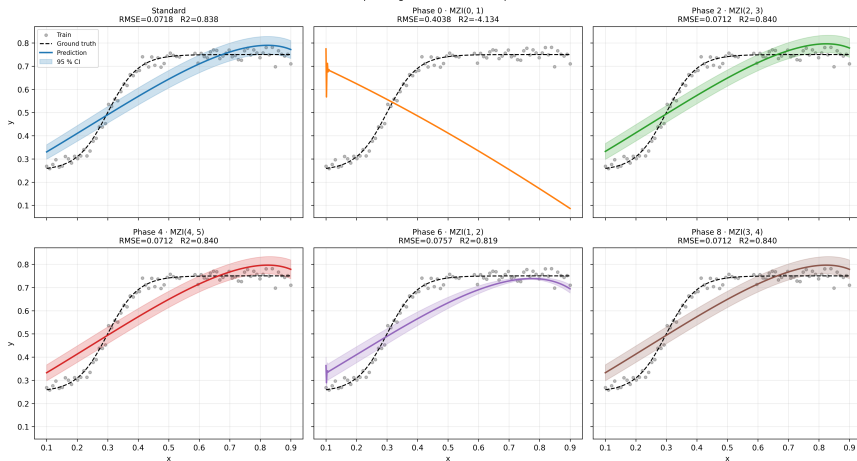
Smooth-Step Placement Sweep

Placement	RMSE	R^2	Coverage
Standard	0.0718	0.8377	0.368
Phase 0	0.4038	-4.1344	0.002
Phase 2	0.0712	0.8403	0.278
Phase 4	0.0712	0.8403	0.278
Phase 6	0.0757	0.8193	0.048
Phase 8	0.0712	0.8403	0.278

- Placement matters a lot.
- Phase 0 is catastrophic.
- Phases 2/4/8 are close to baseline.
- Phase 6 is consistently harmful on smooth step.

Smooth-Step Placement Fits

Smooth step fitting across memristor placements



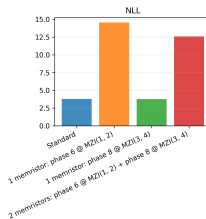
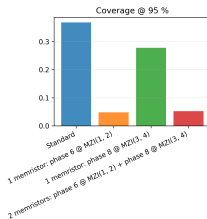
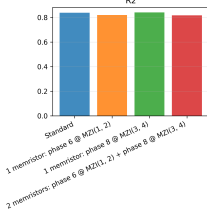
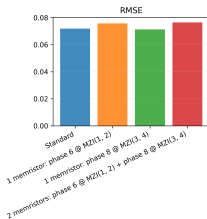
One vs Two Memristors

Config	RMSE	R^2	Coverage	NLL
Standard	0.0718	0.8377	0.368	3.790
Single 6	0.0757	0.8193	0.048	14.559
Single 8	0.0712	0.8403	0.278	3.766
Double 6+8	0.0764	0.8164	0.052	12.595

- Adding a second memristor at phase 6 does not help.
- The best smooth-step result remains standard or single phase 8.
- “More memristors” is not the same as “better fit.”

Multiple-Memristor Comparison

Smooth step metrics: single vs multiple memristors

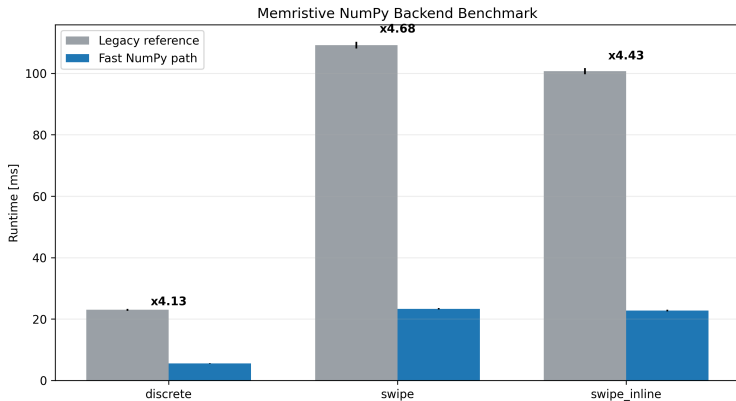


Backend Benchmark

Case	Fast [ms]	Legacy [ms]	Speedup
Discrete	5.562	22.994	4.13x
Swipe	23.315	109.183	4.68x
Swipe inline	22.744	100.722	4.43x

- The new memristive NumPy backend is about 4x–5x faster.
- The speedup is large enough to make rerun studies practical.
- Numerical parity with the old path was preserved by tests.

Benchmark Plot



Final Assessment

Engineering result

The repository is now more reproducible, more modular, and significantly faster for memristive singles reruns.

Scientific result

Memristive feedback is task- and placement-dependent. It can help some targets, but on smooth-step fitting the tested placements and the tested two-memristor pair did not outperform the standard baseline in a robust way.

Next step

Search better multi-memristor pairs, vary memory depth, and test harder functions where temporal-style feedback may be more useful.